
Date: 09/04/2026

Duration: 3 hours

FINAL ASSESSMENT

II YEAR – SEMESTER 4

CSE23AE204 - PCP III

(B.Tech. E01,E02,E03,E05,E06)

SET A

Question 1: Problem Statement

You are developing a **fare calculation module** for a ride-sharing application.

The system processes **one ride request at a time**, provided as a **space-separated string**.
Each ride belongs to a specific category and follows different fare calculation rules.

The system must:

- Identify the ride type from input
- Calculate the total fare based on distance and ride category
- Use **inheritance and polymorphism** to support multiple ride types
- Assign a unique ride ID using a **static variable**
- Return the final fare amount

Fare Rules

Mini Ride

- A base fare is charged.
- An additional per-kilometer charge is applied based on distance.
- The total fare is the sum of base fare and distance charge.

Sedan Ride

- A higher base fare is charged compared to Mini.
- A higher per-kilometer charge is applied.
- The total fare is calculated accordingly.

SUV Ride

- The highest base fare is charged.
- The highest per-kilometer charge is applied.
- The total fare is calculated accordingly.

Example 1

Input

{ "ride": "MINI Rahul 10" }

Output

150.0

Explanation

The base fare and per-kilometer charges applicable to a Mini ride are applied for 10 km.

Question 2: Problem Statement

A set of activities is provided where each activity has a **start time** and **finish time**.

Only **one activity can be performed at a time**.
An activity can be selected only if its start time is **greater than or equal to** the finish time of the previously selected activity.

The goal is to **select the maximum number of non-overlapping activities** that can be performed.

Example 1

Input

start = [1, 3, 0, 5, 8, 5] end = [2, 4, 6, 7, 9, 9]

Output

4

Explanation

Activities after sorting by finish time:

(1,2) (3,4) (5,7) (8,9)

Selected activities:

(1,2) (3,4) (5,7) (8,9)

Maximum activities = **4**

Question 3: Problem Statement

A large manufacturing plant tracks the workload handled by each robotic unit on an assembly line.

You are given an array loads, where:

- loads[i] = number of tasks completed per minute by robot unit **i**

The factory wants to identify a **Load Balance Point (LBP)**:

A position i where:

Sum of loads strictly to the left of i == Sum of loads strictly to the right of i

This helps determine where to place supervisors and load balancers to maintain efficiency.

Your Task

Return the **first Load Balance Point index** if one exists.
If no such point exists, return **-1**.

Input:
loads = [1,7,3,6,5,6]
Output:
3
Explanation:
Left sum = 1 + 7 + 3 = 11
Right sum = 5 + 6 = 11
Index **3** is the Load Balance Point.

Question 4: Problem Statement

You are given a string s.

A **subsequence** is formed by deleting some characters (without changing order).

A subsequence is called **symmetric** if it reads the same forward and backward.

Your task is to find the **length of the longest symmetric subsequence** in the string.

Input:
s = "cbbd"
Output:
2
Explanation:
Subsequence "bb"

Question 5: Problem Statement

A company provides charging facilities for multiple electronic devices throughout the day.

Each device requires charging for a specific **time interval**, represented as: [start, end], where:

- **start** → time when charging begins
- **end** → time when charging finishes

All charging sessions are given as a list of intervals:

intervals = [[start1, end1], [start2, end2], ...]

A **charging port can charge only one device at a time**. If two charging sessions overlap in time, they must use **different ports**.

Your task is to determine the **minimum number of charging ports required** so that all devices can be charged **without any waiting time**.

Input
intervals = [[1,4],[2,5],[7,9]]
Output
2
Explanation
Device charging schedule:
Device1 : [1-----4]
Device2 : [2-----5]
Device3 : [7---9]
Between time **2 and 4**, two devices require charging simultaneously.